Práctica UD4. Automatización de tareas de administración de MariaDB

En esta práctica se trabajará con diferentes mecanismos de automatización de un sistema gestor de bases de datos. Se utilizarán procedimientos almacenados, estructuras de control, disparadores, eventos programados y herramientas del sistema operativo Debian.

El objetivo es desarrollar un conjunto de tareas administrativas automatizadas que permitan mantener, controlar y proteger la información almacenada en MariaDB, reduciendo la necesidad de intervención manual.

> **Entrega obligatoria**
> Se entregará un único archivo PDF que documente todo el trabajo realizado. El documento deberá incluir capturas de pantalla de las operaciones, instrucciones utilizadas, resultados obtenidos y explicaciones de los pasos realizados.
>
> No se entregarán archivos SQL, scripts Bash ni otros archivos independientes. Todo el trabajo deberá quedar documentado dentro del PDF.

1. Preparación del entorno de trabajo

Prepara el entorno de prácticas utilizando la máquina virtual Debian con MariaDB instalado y el cliente MySQL Workbench o la consola de MariaDB.

Realiza las siguientes operaciones:

- Comprueba que el servicio MariaDB está instalado y funcionando.
- Accede al servidor mediante un usuario con los permisos necesarios.
- Crea una base de datos denominada `gestion_automatizada`.
- Comprueba que puedes realizar operaciones SQL sobre ella.
- Prepara las tablas necesarias para los distintos apartados de la práctica.

Utiliza una estructura de datos relacionada con una pequeña empresa, tienda o sistema de gestión. Deberás disponer, como mínimo, de una tabla principal de registros que permita realizar inserciones, modificaciones y eliminaciones.

2. Procedimientos almacenados y variables

Crea procedimientos almacenados que permitan automatizar operaciones habituales sobre la base de datos.

2.1. Procedimiento con variables

- Define un procedimiento que utilice variables locales.
- Utiliza `DECLARE` para declarar las variables.
- Utiliza `SET` para asignar valores.
- Realiza alguna operación de consulta o cálculo con esas variables.
- Ejecuta el procedimiento mediante `CALL`.

2.2. Procedimiento con parámetros

- Crea un procedimiento que reciba al menos un parámetro de entrada.
- Realiza una operación sobre la base de datos utilizando ese parámetro.
- Define otro procedimiento que utilice un parámetro de salida `OUT`.
- Ejecuta los procedimientos y comprueba los resultados.

3. Estructuras de control de flujo

Amplía los procedimientos anteriores incorporando estructuras que permitan controlar la ejecución de las instrucciones.

3.1. Condicionales

- Crea un procedimiento que utilice `IF`, `ELSEIF` y `ELSE`.
- Establece diferentes comportamientos según el valor de una variable o parámetro.
- Comprueba que se ejecutan correctamente las distintas condiciones.

3.2. Bucles

- Desarrolla un procedimiento que utilice un bucle `WHILE`.
- Incorpora un ejemplo utilizando `REPEAT` o `LOOP`.
- Utiliza una condición de salida para evitar ejecuciones indefinidas.
- Comprueba los resultados de las operaciones realizadas.

3.3. Tratamiento de errores

- Incorpora un mecanismo básico de tratamiento de errores en un procedimiento.
- Utiliza un manejador de excepciones mediante `DECLARE HANDLER`.
- Provoca una situación de error controlada y comprueba el comportamiento del procedimiento.

4. Disparadores e integridad de la información

Implementa automatismos que se ejecuten cuando se realicen operaciones sobre las tablas de la base de datos.

4.1. Validación mediante triggers

- Crea un trigger `BEFORE INSERT` que valide los datos antes de insertar un registro.
- Establece una condición que permita rechazar valores incorrectos.
- Utiliza `SIGNAL` para generar un error cuando no se cumpla la condición.
- Comprueba el funcionamiento con una operación válida y otra que deba ser rechazada.

4.2. Auditoría de modificaciones

- Crea una tabla destinada a almacenar el historial de modificaciones.
- Desarrolla un trigger `AFTER UPDATE`.
- Utiliza `OLD` y `NEW` para obtener los valores anteriores y nuevos.
- Registra la fecha de modificación y los datos necesarios para identificar el registro afectado.
- Realiza varias modificaciones y comprueba los registros generados en el historial.

4.3. Registro de eliminaciones

- Crea una tabla para conservar información de los registros eliminados.
- Desarrolla un trigger asociado al evento `DELETE`.
- Utiliza `OLD` para recuperar los valores del registro eliminado.
- Comprueba que la información queda almacenada después de ejecutar un `DELETE`.

5. Eventos programados de MariaDB

Configura el planificador de eventos de MariaDB y utiliza `CREATE EVENT` para automatizar operaciones periódicas.

5.1. Activación del planificador

- Consulta el valor de la variable `event_scheduler`.
- Activa el planificador si se encuentra deshabilitado.
- Configura su activación permanente en Debian.
- Reinicia el servicio MariaDB y comprueba que la configuración se mantiene.

5.2. Creación de eventos

- Crea un evento de ejecución única mediante `ON SCHEDULE AT`.
- Crea un evento periódico utilizando `ON SCHEDULE EVERY`.
- Programa una tarea de mantenimiento que realice una operación sobre los datos.
- Consulta los eventos existentes mediante `SHOW EVENTS`.
- Desactiva y vuelve a activar uno de los eventos.
- Elimina un evento de prueba.

Documenta las fechas, intervalos y operaciones que has configurado. Comprueba el funcionamiento de las instrucciones que forman parte de los eventos.

6. Automatización mediante cron en Debian

Utiliza el planificador `cron` para ejecutar automáticamente scripts SQL desde el sistema operativo.

6.1. Preparación de cron

- Comprueba que el servicio `cron` está instalado y funcionando.
- Consulta las tareas programadas del usuario.
- Prepara un archivo SQL con una operación de mantenimiento.
- Ejecuta el archivo manualmente desde la terminal mediante el cliente `mariadb`.

6.2. Programación de la tarea

- Programa la ejecución del archivo SQL mediante `crontab -e`.
- Configura una periodicidad adecuada.
- Comprueba la entrada creada mediante `crontab -l`.
- Verifica la ejecución y los cambios producidos en la base de datos.

6.3. Registro de ejecuciones

- Redirige la salida de la tarea a un archivo de registro.
- Registra los errores que puedan producirse.
- Consulta los registros del servicio cron.
- Explica cómo comprobarías si una tarea ha terminado correctamente.

7. Copias de seguridad automatizadas

Implementa un sistema de copias de seguridad de una base de datos mediante `mariadb-dump`.

7.1. Copia manual

- Genera una copia de seguridad de la base de datos utilizada en la práctica.
- Guarda el archivo en un directorio específico.
- Incluye la fecha y hora en el nombre del archivo.
- Comprueba que el archivo se ha generado correctamente.

7.2. Script de copia de seguridad

- Crea un script Bash que realice la copia automáticamente.
- Utiliza `mariadb-dump` para exportar la base de datos.
- Comprime el archivo generado.
- Registra la fecha y el resultado de la operación.
- Incorpora un tratamiento básico de errores.
- Configura el acceso a MariaDB mediante un archivo de credenciales protegido.

7.3. Planificación y restauración

- Programa la ejecución periódica del script mediante cron.
- Comprueba que se genera una nueva copia.
- Restaura una de las copias en una base de datos independiente de prueba.
- Comprueba que se han recuperado correctamente las tablas y los datos.

8. Análisis y documentación final

Una vez realizadas todas las operaciones, incluye una valoración técnica del sistema de automatización desarrollado.

En este apartado debes explicar:

- Qué tareas has automatizado y qué objetivo tiene cada una.
- Qué operaciones se ejecutan mediante procedimientos, triggers y eventos.
- Qué tareas se han programado mediante cron.
- Qué diferencias existen entre el planificador de MariaDB y el del sistema operativo.
- Cómo se registran los resultados y se detectan los errores.
- Qué medidas permiten garantizar la integridad de los datos.
- Por qué es necesario comprobar periódicamente las copias de seguridad.

9. Formato y requisitos de entrega

> **Única entrega: documento PDF**
> Se entregará un único archivo PDF con la documentación completa de la práctica.

Contenido obligatorio del PDF

1. **Portada:** título de la práctica, nombre del alumno y fecha.
2. **Preparación:** descripción del entorno y base de datos utilizada.
3. **Desarrollo:** documentación de todos los apartados de la práctica.
4. **Capturas:** evidencias de las instrucciones ejecutadas y de los resultados obtenidos.
5. **Explicaciones:** comentarios propios sobre los pasos realizados y el funcionamiento de cada elemento.
6. **Conclusiones:** valoración final del trabajo realizado.

Requisitos de las capturas

- Las capturas deben mostrar claramente las operaciones realizadas.
- Se deben incluir tanto las instrucciones ejecutadas como sus resultados.
- Las imágenes deben ser legibles y tener un tamaño adecuado.
- Cada captura debe ir acompañada de una explicación de lo que se está realizando.
- No es suficiente con incluir capturas sin comentarios.
- Cuando una operación produzca un error, debe documentarse y explicarse su causa o resolución.

Nombre del archivo

```
UD4_Apellidos_Nombre.pdf
```

10. Consideraciones finales

- Todos los apartados deben estar realizados y documentados.
- Las operaciones deben ejecutarse realmente en el entorno de prácticas.
- Los resultados mostrados en las capturas deben corresponder con las instrucciones explicadas.
- Se valorará especialmente que las explicaciones demuestren comprensión del funcionamiento de cada mecanismo de automatización.
- La presentación, organización y legibilidad del PDF forman parte de la evaluación.